

Sequence of the Upside-Down

Time limit per test: 1 second
Memory limit per test: 256 megabytes
Input: standard input
Output: standard output

The town of Hawkins is in danger. The Demogorgon from the Upside-Down has attacked once again. Our fearless group of friends, Eleven, Mike, Dustin, Max, Will, and Lucas - again went on an adventure to close the gate of the Upside-Down.

However, even Eleven couldn't close the gate with her psychic ability as a peculiar sequence has surfaced. This sequence A, which has a strange connection to the supernatural, contains all natural numbers that avoid the digits **5** and **6**.

A = [1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 13, 14, 17, ...]

For example, the numbers **1234** and **132980** are part of sequence **A**, but the numbers **63214321** and **551** are not part of sequence **A**.

To close the gate to the Upside-Down, the k th number in the series should be entered as a code. But the brave group of friends soon realized it was not a piece of cake to find the k th number of the series.

As the magician of the group (because all programmers are magicians), it's now up to you to go forward and help them and, with Eleven, defeat the Demogorgon to save the world.

Input

The first line of input data contains a single integer t ($1 \leq t \leq 10^4$) - the number of test cases.

In the only line of each test case, there is a single integer k ($1 \leq k \leq 10^{12}$) - the position of the sequence which is the code of the gate.

Output

For each test case, print on a separate line the number a_k in indexing with 1.

Example

input

```
7
3
5
22
10
100
12345
827264634912
```

output

3

7

28

12

144

30091

14023484110040

A sample solution in JAVA

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    static Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int t = sc.nextInt();

        while (t > 0) {
            long n = sc.nextLong();
            ArrayList<Integer> al = new ArrayList<>();
            while (n > 0) {
                int x = (int) (n % 8);
                n = n / 8;

                al.add(x);
            }

            for (int i = al.size() - 1; i >= 0; i--) {
                int x = al.get(i);
                if (x >= 5) x+=2;
                System.out.print(x);
            }
            System.out.println();
            t--;
        }
    }
}
```

